Auditoría Técnica Independiente — Proyecto Artificial Word · 9 de marzo de 2026 · Repositorio: `artificial-word` · Sin dependencia de documentos previos

Auditoría Técnica Independiente — Proyecto Artificial Word

Fecha: 9 de marzo de 2026

Alcance: Repositorio completo (auditoría desde cero, sin depender de documentos previos)

Metodología: Inspección directa del código, análisis de estructura y verificación cruzada

---

Resumen Ejecutivo

Artificial Word es un proyecto multicomponente: un motor de simulación Python (pygame), una demo web Node/React, y varios subproyectos. El motor Python es el componente principal y está bien probado. La pila web es una demo separada con su propio motor de simulación en JavaScript. La documentación es extensa pero fragmentada, con algunas inconsistencias.

Veredicto: El proyecto es técnicamente sólido y el README separa claramente lo real de lo demo. Riesgos principales: estructura dual de Python (`nucleo` / `mundo` vs `core`), uso de `print()` en tests, y dispersión documental.

---

1. Metodología

- **Estructura:** Análisis de directorios, imports y dependencias
- **Motor Python:** Lectura de `principal.py`, `nucleo/simulacion.py`, `sistemas/sistema_persistencia.py`, `cronica_fundacional.py`, `pruebas/run_tests_produccion.py`
- **Web fullstack:** Lectura de `backend/src/server.js`, rutas, `frontend/src/App.jsx`, `config/api.js`
- **Documentación:** Revisión de `docs/`, índices y claims
- **Exports:** Inspección de `IA-Entre-Amigos-main/` y `horizons-export-*/`
- **Riesgos:** Búsqueda de `console.log`, URLs hardcodeadas, `print()`, secretos y referencias rotas

---

2. Mapa de Estructura

| Área | Ruta | Contenido |

|------|------|----------|

| **Motor Python** | `principal.py`, `nucleo/`, `sistemas/`, `acciones/`, `agentes/`, `entidades/`, `mundo/`, `core/`, `interfaz/`, `tipos/`, `utilidades/` | Motor de simulación, persistencia, UI |

| **Backend** | `backend/src/` | API Express, WebSocket, Mission Control, IA, HeroRefuge, DobackSoft |

| **Frontend** | `frontend/src/` | React + Vite, Hub, SimulationView, MissionControl, DobackSoft, FireSimulator |

| **Docs** | `docs/` | 129+ archivos MD, papers, prompts |

| **Subproyecto: IA-Entre-Amigos** | `IA-Entre-Amigos-main/` | Presentación Astro (33 slides sobre IA) |

| **Subproyecto: horizons-export** | `horizons-export-*/` | Export Vite React (estructura de app distinta) |

| **Scripts** | `scripts/`, `iniciar.ps1` | `iniciar_fullstack.ps1`, `iniciar.ps1` |

| **Referencia externa** | `engram-1.7.1/` | Referenciado pero no parte del núcleo |

---

3. Motor Python

3.1 Puntos de entrada

| Archivo | Propósito |

|---------|----------|

| `principal.py` | Entrada principal; `--web` abre landing, `--cronica` ejecuta crónica headless |

| `cronica_fundacional.py` | Ejecución headless independiente; `--seed`, `--ticks`, `--founder`, `--refuge` |

3.2 Módulos principales

- **nucleo/:** `simulacion.py` (orquestador), `gestor_ticks.py`, `bus_eventos.py`, `contexto.py`, `constantes.py`
- **sistemas/:** `sistema_persistencia.py` (SQLite + JSON), `gestor_modo_sombra.py`, `sistema_watchdog.py`, `sistema_regeneracion.py`, `sistema_metricas.py`, `sistema_logs.py`, `sistema_modo_competencia.py`, `cronica_fundacional.py`
- **mundo/:** Usa `core/world/` (ej. `mundo/terreno.py` → `core.world.terreno`)
- **core/:** `world/`, `shelter/`, `entity/`, `simulation/` — implementación de bajo nivel
- **agentes/:** `memoria.py` reexporta desde `systems/memory/memoria_entidad.py`

3.3 Persistencia

- `sistemas/sistema_persistencia.py`: SQLite `mundo_artificial.db`, backup JSON, guardado automático cada N ticks
- Tablas: `estado` (id, tick, datos, actualizado_at)

3.4 Tests

- **Runner:** `pruebas/run_tests_produccion.py` — 11 suites
- **Suites:** `test_estructural`, `test_core`, `test_cronica_fundacional`, `test_modo_sombra_completo`, `test_perseguir_hasta_matar`, `test_interacciones_sociales`, `test_bug_robar`,

`test_watchdog_fixes`, `test_watchdog_integracion`, `test_arranque_limpio`, `test_integracion_produccion`

- **Entorno:** `SDL_VIDEODRIVER=dummy`, `SDL_AUDIODRIVER=dummy` para headless

3.5 Veredicto

- **Ejecutable:** Sí — `python principal.py` y `python cronica_fundacional.py --ticks 200` son puntos de entrada válidos
- **Tests:** 11 suites definidos
- **Estructura dual:** `nucleo` / `mundo` vs `core` — `mundo` envuelve `core`; estructura redundante pero consistente

---

4. Web Fullstack

4.1 Backend (Node.js Express)

- **Puerto:** 3001 (configurable vía `PORT`)
- **Rutas:** `/api` (simulación), `/api/ai`, `/api/hero`, `/api/dobacksoft`, `/api/subscription`, `/api/admin`, `/api/chess`, `/api/mission-control`, `/api/approvals`, `/api/boards`, `/api/gateways`
- **WebSocket:** `/ws` para estado de simulación y Mission Control
- **Simulación:** `backend/src/simulation/engine.js` — motor JS, no Python

4.2 Frontend (React + Vite)

- **Puerto:** 5173
- **Enrutamiento:** Basado en hash (`#hub`, `#simulation`, `#missioncontrol`, etc.)
- **Vistas principales:** LandingPublic, Landing, Hub, SimulationView, MinigamesLobby, DobackSoft, FireSimulator, MysticQuestView, MissionControl, AdminPanel, Docs

4.3 Qué funciona

| Componente | Estado | Notas |

|----------|--------|-------|

| Hub | Funcional | Secciones, pilares, navegación |

| SimulationView | Funcional | Usa motor JS del backend, WebSocket |

| Mission Control | Funcional | Runtime seeded, boards, approvals, gateways |

| HeroRefuge | Funcional | Persistencia SQLite, scoping por `playerId` |

| DobackSoft | Demo | `sessionsStore` en memoria, `MOCK_ROUTE` |

| FireSimulator | Demo | Reproduce datos del visor; sin telemetría real |

4.4 Configuración

- `frontend/src/config/api.js`: Usa `VITE_BACKEND_PORT`, `VITE_API_BASE_URL`; sin localhost hardcodeado

- Sin `console.log` en frontend ni backend (cumple AGENTS.md)

4.5 Veredicto

- La pila web es una demo separada con su propio motor de simulación JS
- README y `docs/MODOS_EJECUCION.md` indican correctamente que la web no es el motor Python

---

5. Documentación

5.1 Volumen y organización

- 129+ archivos MD en `docs/`
- `INDEX_DOCUMENTACION.md` lista lo esencial; `TABLA_DE_CLAIMS_AW.md` apunta a `DOCUMENTO_UNIFICADO_AW.md`
- Varios documentos "definitivos": `DOCUMENTO_FINAL.md`, `DOCUMENTO_UNICO.md`, `DOCUMENTO_UNIFICADO_AW.md`

5.2 Consistencia

- **README:** Separación clara entre real y demo; golden path = Python
- **MODOS_EJECUCION.md:** Alineado con README
- **REALIDAD_VS_VISION.md:** Clasificación en cinco categorías
- **GOLDEN_PATH.md:** Referencia a `iniciar.ps1`; existe y delega en `scripts/iniciar_fullstack.ps1` para web

5.3 Posibles problemas

- **Inconsistencia de rutas:** `docs\` vs `docs/` en referencias (Windows vs Unix)
- **Referencias rotas:** `INDEX_DOCUMENTACION.md` referencia `ARTIFICIAL_WORLD_COMPLETO.pdf`; solo se encuentran `.html` y `.md`
- **Solapamiento:** Varios documentos "definitivos" pueden generar confusión

---

6. Exports

6.1 IA-Entre-Amigos-main

- **Stack:** Astro 5.17.1
- **Contenido:** Workshop de IA de 33 slides (fundamentos, arquitecturas, agentes, MCP)
- **Relación:** Presentación independiente; sin código compartido con el proyecto principal
- **Ejecución:** `npm run dev` en `IA-Entre-Amigos-main/`

6.2 horizons-export-*

- **Stack:** Vite 7.3.1, React 18.3.1
- **Estructura:** `apps/web/` con `HomePage`, `PaperPage`, `HubPage`, `GamesPage`, `FireSimulator`

- **Componentes:** `ArtificialWorldSimulator`, `EcosistemaSection`, `ConceptoSection`, `CTASection`, `CVSection`, `ScientificPaperSection`, etc.
- **Relación:** Export/build separado de un sitio de marketing/landing de Artificial World; enrutamiento y estructura distintos al frontend principal

---

7. Matriz de Riesgos

| Riesgo | Severidad | Probabilidad | Mitigación |

|--------|----------|----------|----------|

| Sobreventa (web = motor Python) | Alta | Baja | README y docs ya lo aclaran; mantener |

| Estructura dual Python (nucleo/core) | Media | N/A | Refactorizar o documentar claramente |

| `print()` en tests Python | Baja | Alta | AGENTS.md prefiere logger; tests pueden estar exentos |

| Desajuste env cupón | Baja | Media | `.env.example` tiene `FUNDADOR_COUPON`; store usa `DOBACKSOFT_COUPON_CODE` o `FUNDADOR1000` |

| Enlaces rotos en docs | Baja | Media | Auditar enlaces; corregir o eliminar |

| Dispersión documental | Media | Alta | Consolidar docs "definitivos"; punto de entrada único |

| DobackSoft mock presentado como real | Media | Baja | Docs indican demo; asegurar que etiquetas UI coincidan |

---

8. Recomendaciones

1. **Python:** Sustituir `print()` en código no-test por logging; documentar política de `print()` en tests en AGENTS.md.
2. **Documentación:** Elegir un único documento "definitivo" y redirigir los demás; corregir o eliminar enlaces rotos (ej. `ARTIFICIAL_WORLD_COMPLETO.pdf`).
3. **Config cupón:** Alinear `.env.example` con `DOBACKSOFT_COUPON_CODE` y documentar valores esperados.
4. **Estructura:** Añadir nota breve de arquitectura explicando `nucleo`/`mundo` vs `core` y sus roles.
5. **Exports:** Añadir README en `horizons-export-*/` e `IA-Entre-Amigos-main/` describiendo propósito y relación con el repo principal.

---

9. Veredicto Final

ESTADO: APROBADO (con observaciones)

El proyecto es técnicamente coherente. El motor Python es el componente principal y está bien probado. La pila web se describe correctamente como demo con su propio motor. La documentación es sólida en realidad vs visión pero está fragmentada. No se encontraron problemas críticos de seguridad; la API usa variables de entorno y no hay `console.log` ni URLs hardcodeadas en frontend/backend. Los principales seguimientos son consolidación documental y limpiezas menores en Python/config.

Auditoría realizada de forma independiente. Sin dependencia de DOCUMENTO_UNIFICADO_AW.md ni documentos de auditoría previos.

Auditoría realizada de forma independiente. Sin dependencia de DOCUMENTO_UNIFICADO_AW.md ni documentos de auditoría previos. Fecha: 9 de marzo de 2026.